

Home task: data engineering

Phase 0: Preparation and Financial Safety

1. AWS Free Tier Account Creation

An AWS Free Tier account was successfully created using an email address and a virtual Monobank card with a minimal balance for verification purposes.

To improve account security and avoid unexpected charges:

- Multi-Factor Authentication (MFA) was enabled.
- AWS Budget alerts were configured to monitor spending.
- Free Tier limitations were reviewed before using AWS services.

2. Billing Alarm Configuration

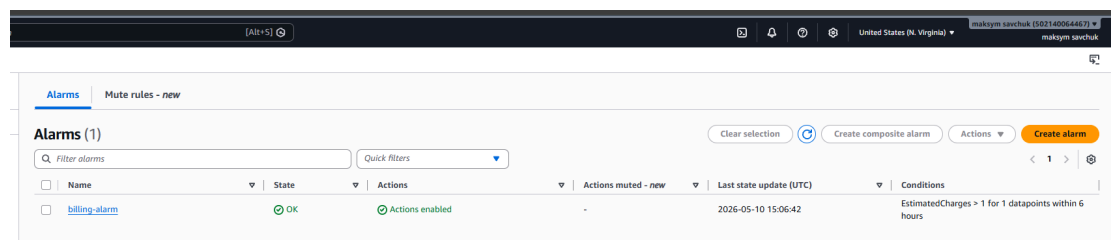
A CloudWatch Billing Alarm was created to monitor AWS charges.

The alarm was configured with the following condition:

- `EstimatedCharges > 1 USD`

Amazon SNS email notifications were also configured to send billing alerts to the registered email address.

The SNS subscription was confirmed successfully via email confirmation link.



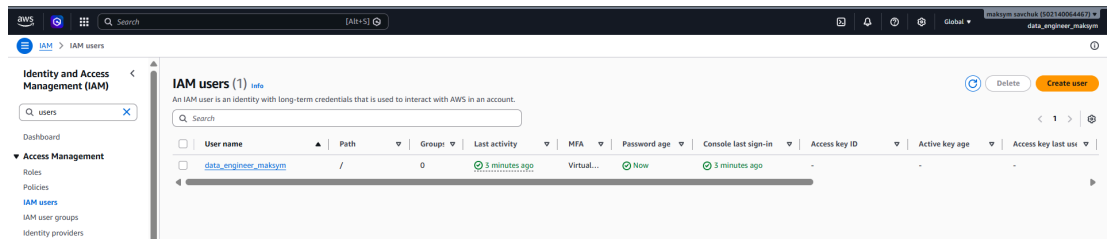
Phase 1: Identity and Access Management (IAM)

A new IAM user was created for working with AWS services instead of using the root account.

The user was named `data_engineer_maksym`.

The following security and access settings were configured:

- AWS Management Console access was enabled.
- The `AdministratorAccess` policy was attached to the user.
- Multi-Factor Authentication (MFA) was enabled.
- The first sign-in was completed successfully using the IAM user account.

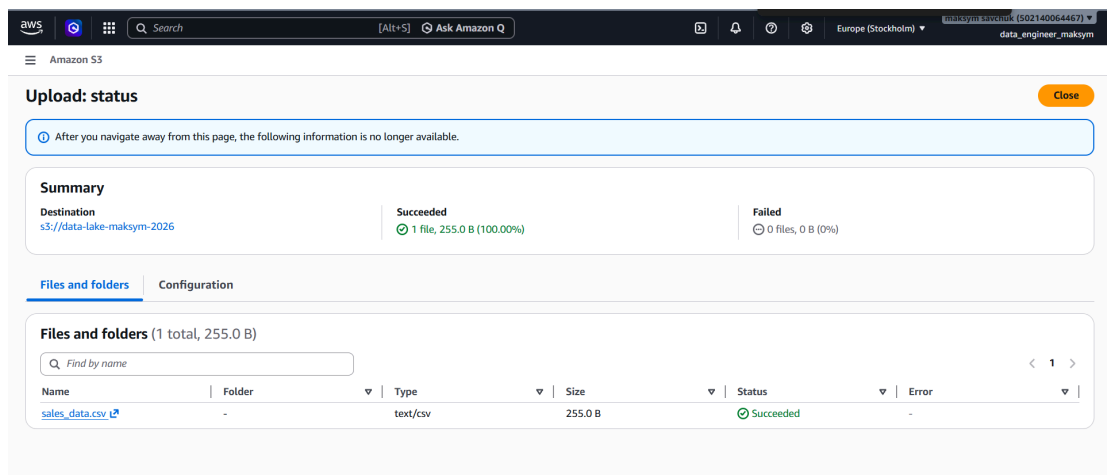


Phase 2: Building a Data Lake with S3, Glue and Athena

2.1 S3: Data Storage

An S3 bucket named `data-lake-maksym-2026` was created to store the source data for the Data Lake pipeline.

A CSV file named `sales_data.csv` was uploaded to the bucket.



2.2 IAM Role: Access for AWS Glue

An IAM role named `glue-s3-role-maksym` was created for AWS Glue.

This role allows AWS Glue to access the uploaded CSV file in the S3 bucket and create metadata in the Glue Data Catalog.

The following policies were attached to the role:

- `AmazonS3FullAccess`
- `AWSGlueServiceRole`

This role will be used later by the Glue Crawler to scan the S3 data and automatically create a table schema.

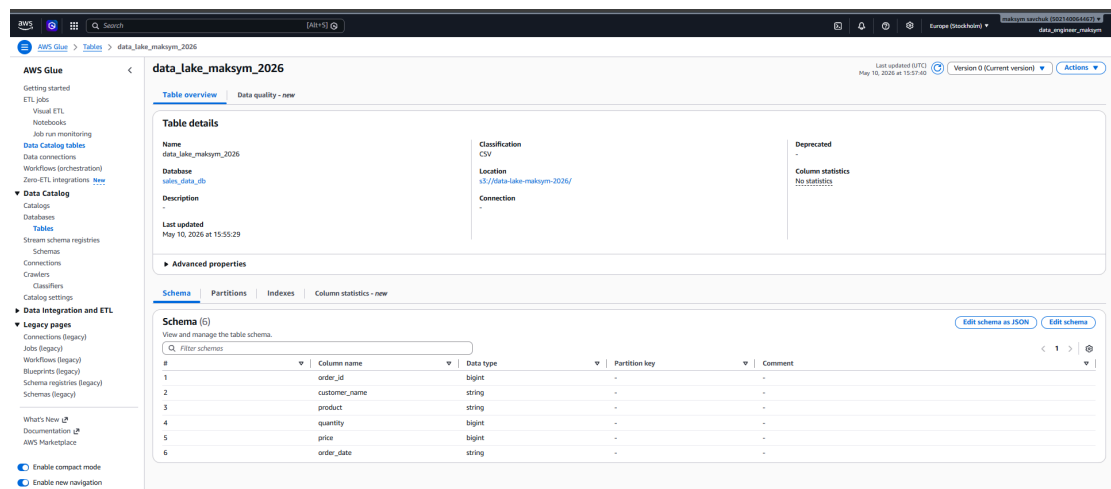
2.3 AWS Glue Crawler and Data Catalog

An AWS Glue Crawler named `sales-data-crawler` was created to scan the CSV file stored in the S3 bucket.

The crawler used the IAM role `glue-s3-role-maksym` and the target database `sales_data_db`. After running the crawler, AWS Glue automatically detected the schema of the uploaded file and created a table in the Glue Data Catalog.

The resulting table was named `data_lake_maksym_2026`. The schema includes the following columns:

- `order_id` — `bigint`
- `customer_name` — `string`
- `product` — `string`
- `quantity` — `bigint`
- `price` — `bigint`
- `order_date` — `string`



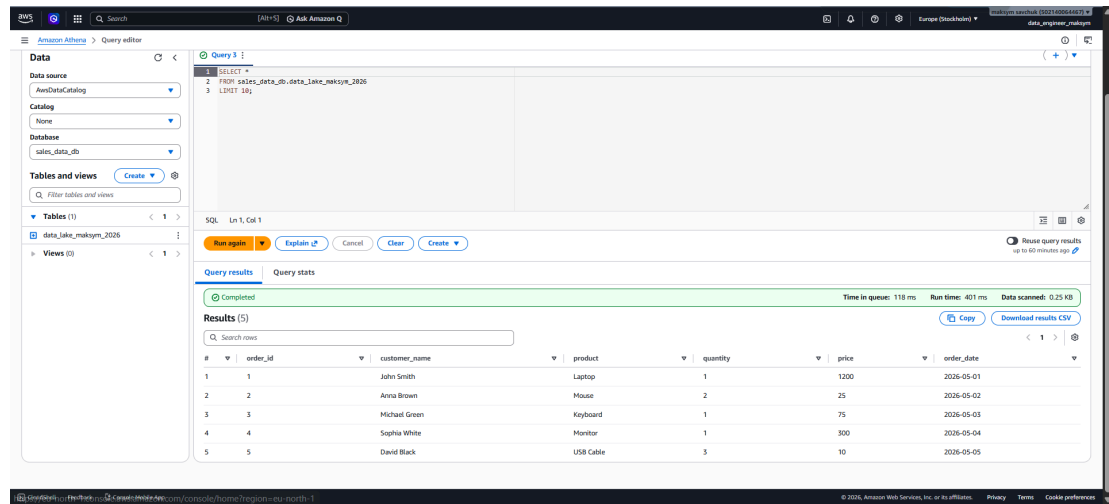
2.4 Amazon Athena: Querying Data

Amazon Athena was used to query the table created by AWS Glue Crawler.

The database `sales_data_db` and the table `data_lake_maksym_2026` were selected in the Athena Query Editor.

The following SQL query was executed:

```
SELECT *
FROM sales_data_db.data_lake_maksym_2026
LIMIT 10;
```



Phase 3: LocalStack and AWS CLI (Optional)

As an optional part of the task, LocalStack was used to simulate AWS services locally through Docker and AWS CLI.

LocalStack was started in a Docker container using the image

`localstack/localstack:4.4.0`.

After the service became ready, AWS CLI commands were executed with the LocalStack endpoint:

```
aws --endpoint-url=http://localhost:4566 s3 mb s3://test-bucket
aws --endpoint-url=http://localhost:4566 s3 cp data.csv s3://test-bucket
aws --endpoint-url=http://localhost:4566 s3 ls s3://test-bucket
```

```
LocalStack version: 4.4.0
LocalStack build date: 2025-05-08
LocalStack build git hash: 21f6e5fb2

Ready.
2026-05-10T16:37:06.541 INFO --- [et.reactor-0] localstack.request.aws : AWS s3.ListBuckets => 200
2026-05-10T16:37:14.798 INFO --- [et.reactor-0] localstack.request.aws : AWS s3.CreateBucket => 200
2026-05-10T16:37:32.449 INFO --- [et.reactor-0] localstack.request.aws : AWS s3.PutObject => 200
2026-05-10T16:37:40.603 INFO --- [et.reactor-0] localstack.request.aws : AWS s3.ListObjectsV2 => 200
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\User> wsl
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ export AWS_ACCESS_KEY_ID=test
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ export AWS_SECRET_ACCESS_KEY=test
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ export AWS_DEFAULT_REGION=us-east-1
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 ls
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 mb s3://test-bucket
make_bucket: test-bucket
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ cat > data.csv <<EOF
id,name,product,price
1,John,Laptop,1200
2,Anna,Mouse,25
3,Mark,Keyboard,75
EOF
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 cp data.csv s3://test-bucket
upload: ./data.csv to s3://test-bucket/data.csv
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$ aws --endpoint-url=http://localhost:4566 s3 ls s3://test-bucket
2026-05-10 16:37:32    76 data.csv
user@LAPTOP-H03GEGCQ:/mnt/c/Users/User$
```